Report: Imitation Learning

Objective

The primary objective of this assignment was to develop a driving policy for the CarRacing-v3 environment by leveraging expert demonstrations. Initially, Behavioral Cloning (BC) was employed to learn the policy. However, pure Behavioral Cloning often suffers from covariate shift, where the learned policy diverges from the expert's distribution, leading to compounding errors. To mitigate this, the Dataset Aggregation (DAgger) algorithm was introduced as an iterative method to refine the policy by continuously collecting and relabeling states encountered by the learner.

Environment and Dataset

The policy was trained and evaluated within the Gymnasium CarRacing-v3 environment, which is a discrete action space environment. It provides 5 distinct actions: "do nothing", "steer left", "steer right", "gas", and "brake".

Image preprocessing was crucial for handling the visual observations from the environment. The raw observations underwent the following transformations:

- **Cropping**: Images were cropped to a shape of (84, 96).
- **Resizing**: The cropped images were resized to (84, 84) pixels.
- **Grayscale Conversion**: The images were converted to grayscale, resulting in a single channel observation.
- **Normalization**: Pixel values were normalized to the range [0, 1] by dividing by 255.0.
- **Frame Stacking**: Four consecutive preprocessed frames were stacked to form a single observation, providing temporal context to the agent.

Neural Network Architecture

The neural network architecture, named PolicyNetwork, was designed to process image-based observations and output action logits. Convolutional Neural Networks (CNNs) were chosen due to their efficacy in image feature extraction. The architecture consisted of:

- **Input Layer**: A single-channel image of size (1, 84, 84).
- **Convolutional Layers**: Three 2D convolutional layers with ReLU activation functions:
    - conv1: nn.Conv2d(1, 32, kernel_size=8, stride=4)
    - conv2: nn.Conv2d(32, 64, kernel_size=4, stride=2)
    - conv3: nn.Conv2d(64, 64, kernel_size=3, stride=1)

- **Flattening**: The output of the last convolutional layer, which had a size of (64, 7, 7), was flattened into a single vector of 64 * 7 * 7 = 3136 features.
- **Fully Connected Layer**: A linear layer nn.Linear(3136, n_units_out), where n_units_out was 5, corresponding to the five discrete actions in the environment.
- **Output**: Logits for each action, which were then passed through a Categorical distribution to sample actions.

Behavioral Cloning Implementation

Behavioral Cloning was framed as a supervised learning problem. The goal was to train the PolicyNetwork to predict the expert's action given a state.

- **Loss Function**: nn.CrossEntropyLoss() was used, appropriate for multi-class classification where actions are discrete categories.
- **Optimizer**: The Adam optimizer (torch.optim.Adam) was employed with an initial learning rate of 1e-4.
- **Batch Size**: Training was conducted with a batch_size of 64.
- **Training Procedure**: The model was trained for 20 epochs. In each epoch, the training data was iterated, gradients were computed, and weights updated. The logger tracked training_loss.
- **Validation Loop**: After each training epoch, the model was evaluated on a separate validation set. The validation_loss and validation_accuracy were calculated to monitor generalization performance.
- **Model Checkpointing**: The model's state dictionary was saved using torch.save(model.state_dict(), logger.param_file)whenever a new best validation accuracy was achieved.
- **ONNX Export**: The final trained model, specifically the best performing model based on validation accuracy, was exported to the ONNX format using save_as_onnx(model, sample_state, logger.onnx_file). This ensured compatibility with the evaluation script.
- **Accuracy Tracking**: Validation accuracy was used as the primary metric for model selection and logging, with the highest accuracy determining the best_val_accuracy`.

DAgger Implementation

DAgger (Dataset Aggregation) was implemented to overcome the limitations of pure BC by iteratively refining the policy and addressing covariate shift. The core DAgger loop consisted of:

- **Policy Mixing (Beta Annealing)**: A beta_schedule linearly decayed from 1.0 (full expert control) to 0.1 (mostly learner control) over 10 DAgger rounds. In each step of an episode, random.random() < betadetermined whether the expert policy or the current train_policy_dagger selected the action.
- **Trajectory Collection**: During an episode, if the train_policy_dagger chose an action, the observed state (current_stacked_state[-1]) was collected.
- **Expert Relabeling**: For each collected state (where the learner chose the action), the expert's action (expert_policy.select_action(current_stacked_state)) for that specific stacked state was also recorded. This provided

ground truth labels for the learner's visited states.

- **Aggregation of New Samples**: The DemonstrationDataset.append() method was used to add the newly collected states and their expert-relabel actions to the dagger_train_set, effectively expanding the training dataset with states encountered by the current policy.
- **Iterative Retraining**: After each DAgger round (i.e., after collecting new demonstrations), the model was fine-tuned for epochs_per_dagger_round (5 epochs) using the aggregated dataset (dagger_train_set). This ensured the policy learned from both original expert data and new states it explored.

Training Configuration and Parameter Choices

The following parameters were used throughout the training process:

- **Batch Size**: 64 for both BC and DAgger training.
- **Optimizer**: Adam optimizer.
- **Learning Rate**: 1e-4 for Behavioral Cloning and 1e-5 for DAgger fine-tuning (a smaller LR for fine-tuning is common).
- **Loss Function**: nn.CrossEntropyLoss().
- **Epochs**: 20 epochs for Behavioral Cloning. Each DAgger round involved 5 epochs of retraining.
- **DAgger Iterations**: 10 DAgger rounds.
- **Beta Values**: A linear beta schedule from 1.0 to 0.1 over 10 DAgger rounds (np.linspace(1.0, 0.1, DAgger_rounds)).
- **Device Selection**: Training and inference were performed on cuda if a GPU was available, otherwise on cpu.
- **Dataset Handling**: DemonstrationDataset with num_workers=2for data loading and pin_memory=True for faster data transfer to GPU.

- **Model Checkpointing**: The model with the best validation accuracy was saved after each epoch (BC) or DAgger round (DAgger).

Evaluation Method

Evaluation of the trained agents was performed directly within the CarRacing-v3 environment, simulating real-world interaction:

- **Environment Setup**: A fresh environment was created using make_envwith optional video recording for qualitative analysis.
- **Agent Interaction**: The Agent class wrapped the PolicyNetwork to provide a select_action method. This method received a state, normalized it, passed it through the model to get logits, and then sampled an action using a Categorical distribution (stochastic action sampling).
- **Episode Runs**: The run_episode function executed a single episode, recording the score. To account for the inherent stochasticity of the environment and policy, agents were evaluated over multiple episodes (n_eval_episodes = 10). The mean score and standard deviation were reported to provide a more robust measure of performance.
- **ONNX-based Evaluation Pipeline**: The final models (both BC and DAgger) were exported to ONNX format. This ONNX model could then be loaded and used by external evaluation scripts, ensuring compatibility with the assignment's submission requirements.

Results and Observations

Behavioral Cloning Performance: During Behavioral Cloning training over 20 epochs, the validation accuracy gradually improved from approximately 0.7054 to a peak of 0.7116 (Epoch 9). The mean score over 10 evaluation episodes for the BC agent was approximately 772.85 (observed from a single run output) and Mean Score: 792.80 (Std: 37.95) (from 10 episodes).

Improvements after DAgger: The DAgger algorithm was applied for 10 rounds, with 5 retraining epochs per round. The dagger_train_set expanded with each round, starting from 94284 samples and reaching 98720 samples by the end of Round 10. While the validation accuracy on the original validation set remained around 0.708-0.710, the DAgger process aimed to improve performance in the actual environment by reducing covariate shift. The mean score over 10 evaluation episodes for the DAgger-trained agent was Mean Score: 785.40 (Std: 92.51). This shows a slight drop in mean score but higher standard deviation, which might indicate some instability or higher exploration. However, the intent of DAgger

is to improve real-world performance by training on learner-visited states, which isn't always reflected in validation accuracy on the original, fixed validation set.